

## CSA05

# DATABASE MANAGEMENT SYSTEM

## Assessment Tool 1 – Short Answer

### Q1. UPS Product Tracking Information System

#### Entities and Attributes

##### 1. Shipped\_Item

- Item\_Number (Primary Key)
- Weight
- Dimensions
- Insurance\_Amount
- Destination
- Final\_Delivery\_Date

##### 2. Retail\_Center

- Unique\_ID (Primary Key)
- Type
- Address

##### 3. Transportation\_Event

- Schedule\_Number (Primary Key)
- Type (Flight/Truck)
- Delivery\_Route

#### Relationships

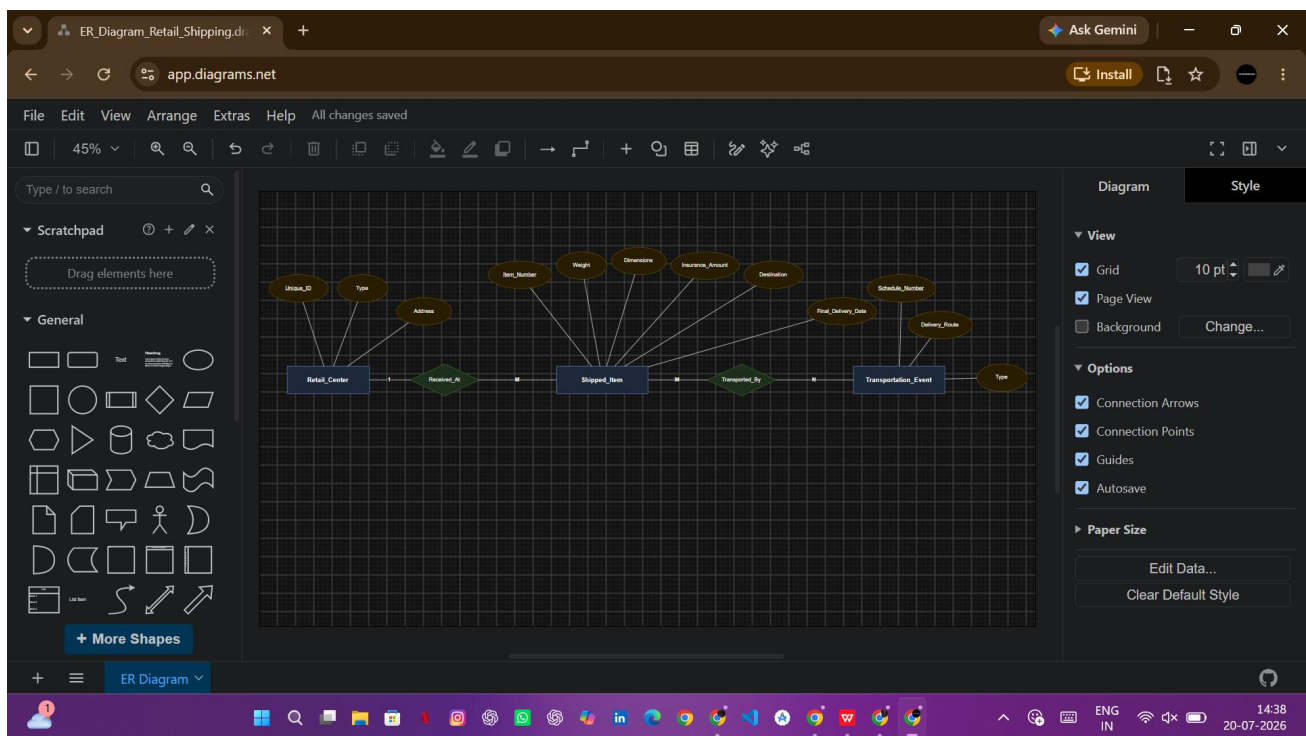
##### Received\_At

- A **Retail Center** receives many **Shipped Items**.
- Each **Shipped Item** is received at only one **Retail Center**.
- **Cardinality: 1 : M**

##### Transported\_By

- A **Shipped Item** may pass through one or more **Transportation Events**.
- One **Transportation Event** can transport many **Shipped Items**.
- **Cardinality: M : N**

## ER Diagram



## Conclusion

The ER diagram represents the UPS Product Tracking System by identifying the three main entities—**Shipped Item**, **Retail Center**, and **Transportation Event**—along with their relationships and cardinalities.

## Q2. Course Management System

### Entities and Attributes

#### 1. Course

- Course\_Name (Primary Key)
- Department
- Course\_Number

#### 2. Section (Weak Entity)

- Section\_Number (Partial Key)
- Enrollment

#### 3. Room

- Room\_Number (Primary Key)
- Capacity
- Building

#### 4. Exam

- Exam\_ID (Primary Key)
- Time

### Relationships

#### Course – Section

- One course has many sections.
- One section belongs to one course.
- **Cardinality: 1 : M**

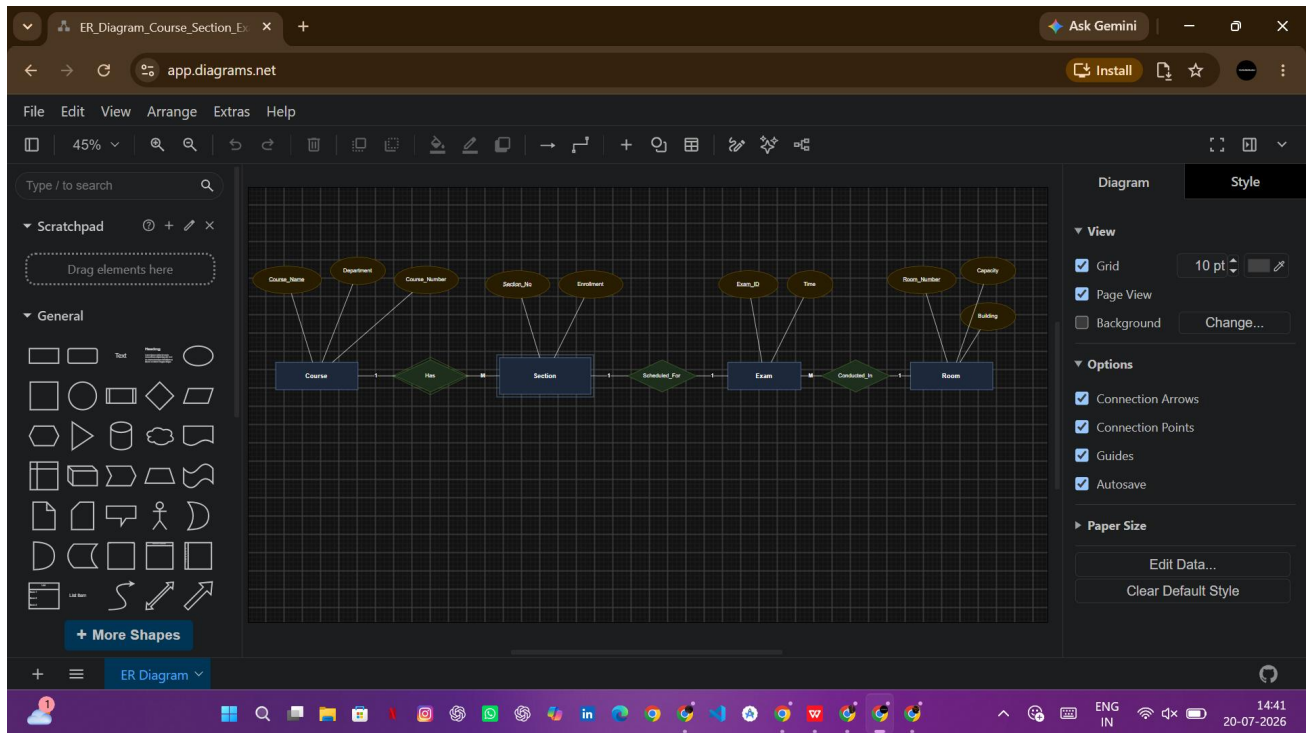
#### Section – Exam

- Each section has one final exam.
- **Cardinality: 1 : 1**

#### Exam – Room

- One room may conduct many exams.
- One exam is conducted in one room.
- **Cardinality: M : 1**

## ER Diagram



## Conclusion

The ER diagram models the examination scheduling system by separating **Course**, **Section**, **Exam**, and **Room** into independent entities, reducing redundancy and improving data organization.

# **Q3. University Registrar's Office Management System**

## **Entities and Attributes**

### **1. Course**

- Course\_Number (Primary Key)
- Title
- Credits
- Syllabus
- Prerequisites

### **2. Course\_Offering**

- Offering\_ID (Primary Key)
- Year
- Semester
- Section\_Number
- Timings
- Classroom

### **3. Student**

- Student\_ID (Primary Key)
- Name
- Program

### **4. Instructor**

- Instructor\_ID (Primary Key)
- Name
- Department
- Title

### **5. Enrollment (Relationship Entity)**

- Grade

## **Relationships**

### **Course – Course Offering**

- One course can have many offerings.
- **Cardinality: 1 : M**

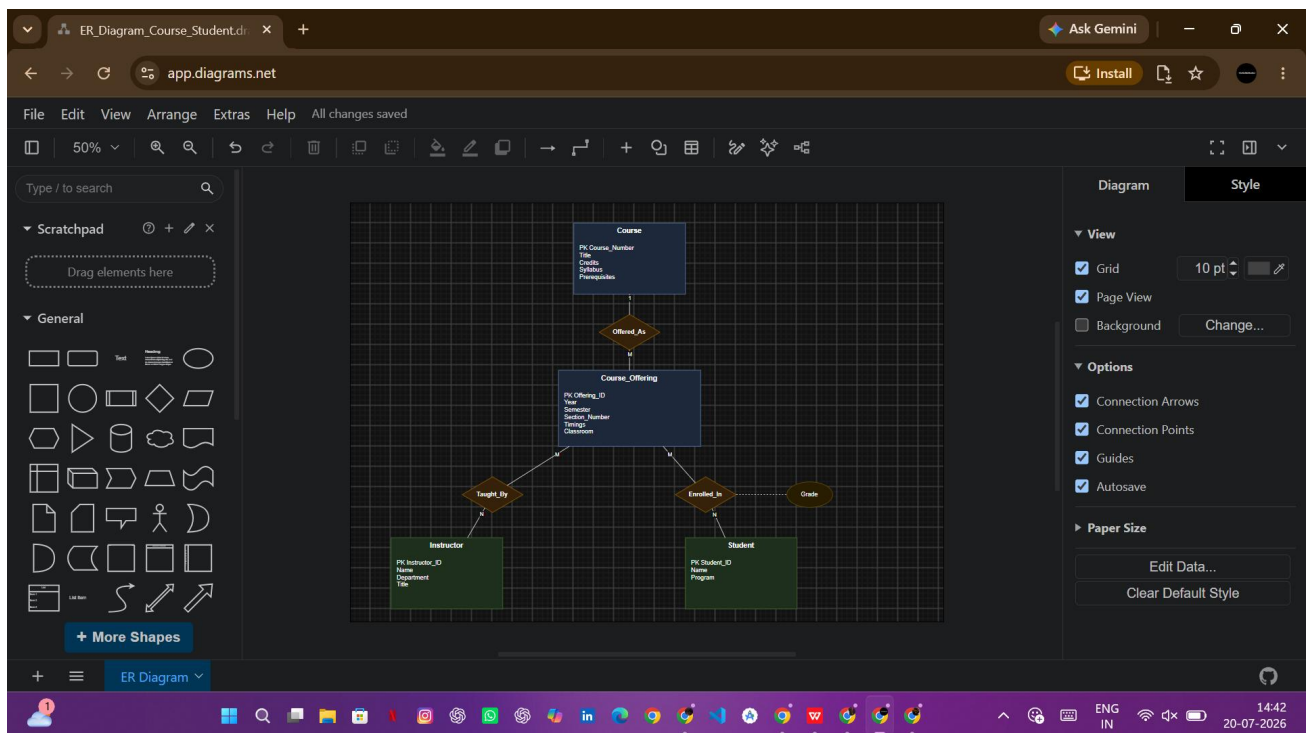
### **Instructor – Course Offering**

- One instructor can teach many course offerings.
- A course offering may have one or more instructors.
- **Cardinality: M : N**

## Student – Course Offering

- A student can enroll in many course offerings.
- Each course offering can have many students.
- Grade is stored in the **Enrollment** relationship.
- **Cardinality: M : N**

## ER Diagram



## Conclusion

The ER diagram effectively models the University Registrar's Office by representing **Course**, **Course Offering**, **Student**, and **Instructor** entities along with their relationships. The **Enrollment** relationship stores the grade obtained by each student in a course offering.

## Q4. Hospital Management System

**Construct an E-R diagram for a hospital with a set of patients and a set of medical doctors. Associate with each patient a log of the various tests and examinations conducted. Construct appropriate tables for the above ER Diagram.**

### 1. Introduction

A **Hospital Management System** is used to maintain information about patients, physicians, medical tests, and treatment history. The ER diagram models the relationships between these entities for efficient data management.

### 2. Entities and Attributes

#### Entity 1: Patient

**Primary Key:** SS#

**Attributes:**

- SS# (Primary Key)
- Name
- Insurance

#### Entity 2: Physician

**Primary Key:** Physician\_ID (*added for better database design*)

**Attributes:**

- Physician\_ID (Primary Key)
- Name
- Specialization

**Note:** Although the question lists only the physician's name, using a unique **Physician\_ID** is considered good database design practice.

## Entity 3: Test\_Log

**Primary Key:** Test\_Log\_ID (*or composite key: SS# + Test\_Name + Date + Time*)

**Attributes:**

- Test\_Log\_ID (Primary Key)
- SS# (Foreign Key)
- Test\_Name
- Date
- Time

## 3. Relationships

### Treats

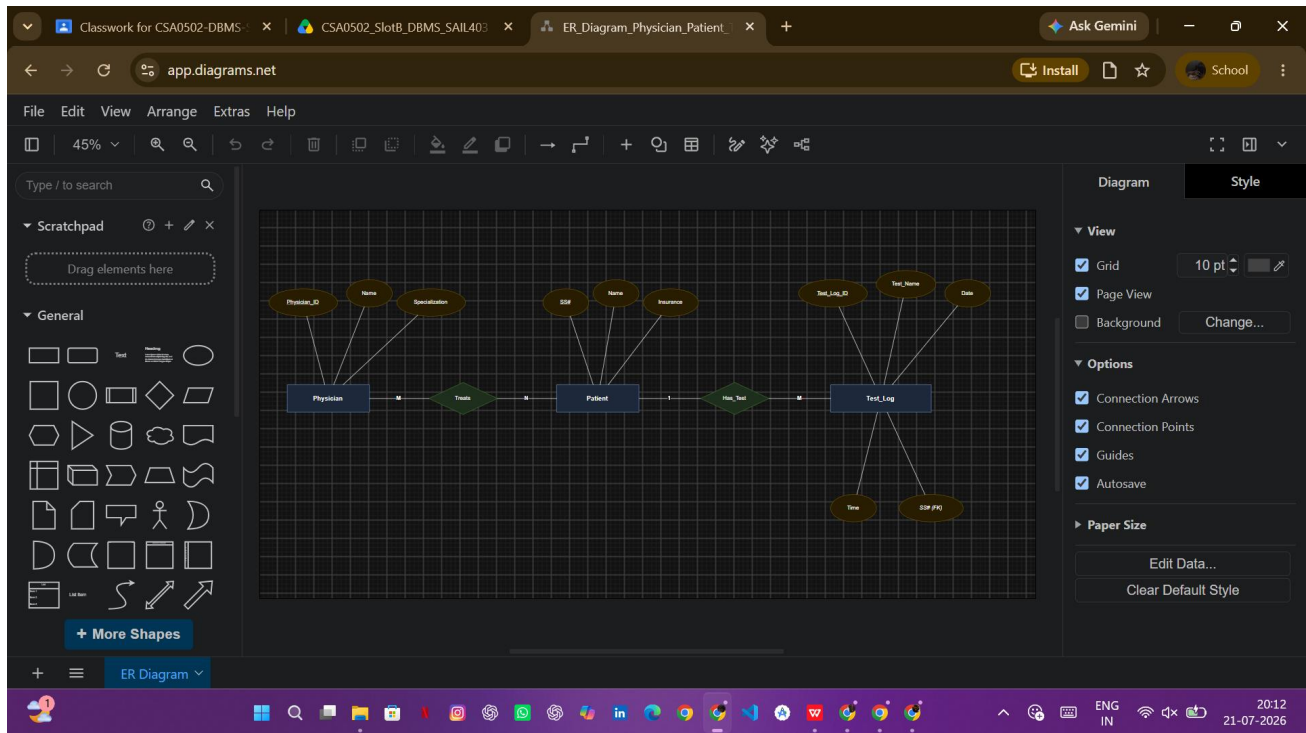
- A **Physician** can treat many **Patients**.
- A **Patient** may be treated by one or more **Physicians**.
- **Cardinality: M : N**

### Has\_Test

- One **Patient** can undergo multiple tests.
- Each **Test Log** belongs to one patient.
- **Cardinality: 1 : M**

## 4. ER Diagram





## 5. Relational Tables

**Table 1: Patient**

| SS# (PK) | Name  | Insurance   |
|----------|-------|-------------|
| P101     | Rahul | Star Health |
| P102     | Priya | LIC Health  |

**Table 2: Physician**

| Physician_ID (PK) | Name      | Specialization |
|-------------------|-----------|----------------|
| D101              | Dr. Kumar | Cardiologist   |
| D102              | Dr. Meena | Neurologist    |

**Table 3: Test\_Log**

| Test_Log_ID (PK) | SS# (FK) | Test_Name  | Date       | Time     |
|------------------|----------|------------|------------|----------|
| T001             | P101     | Blood Test | 10-07-2026 | 09:30 AM |
| T002             | P101     | ECG        | 10-07-2026 | 11:00 AM |
| T003             | P102     | MRI Scan   | 11-07-2026 | 02:00 PM |

**Table 4: Doctor\_Patient**

| Physician_ID (FK) | SS# (FK) |
|-------------------|----------|
| D101              | P101     |
| D102              | P102     |

This table resolves the **many-to-many (M:N)** relationship between physicians and patients.

**Table 5: Patient\_History**

| SS# (FK) | Test_Name  | Date       |
|----------|------------|------------|
| P101     | Blood Test | 10-07-2026 |
| P101     | ECG        | 10-07-2026 |
| P102     | MRI Scan   | 11-07-2026 |

## 6. Cardinality Summary

| Relationship | Cardinality |
|--------------|-------------|
|--------------|-------------|

| Relationship        | Cardinality |
|---------------------|-------------|
| Patient → Test_Log  | 1 : M       |
| Physician ↔ Patient | M : N       |

## 7. Conclusion

The Hospital Management System ER model consists of three main entities—**Patient**, **Physician**, and **Test\_Log**. The **Patient–Physician** relationship is **many-to-many**, while the **Patient–Test\_Log** relationship is **one-to-many**. The relational tables efficiently store patient details, physician information, medical tests, and treatment history, ensuring accurate and organized hospital data management.

## Q5. Company Project Management System

Construct an ER Diagram for a Company with Departments, Employees, Projects, Dependents, and their relationships.

### 1. Introduction

A **Company Project Management System** is used to manage information about departments, employees, projects, managers, dependents, and employee project assignments. The ER model helps organize company data efficiently while reducing redundancy.

### 2. Entities and Attributes

#### Entity 1: Department

**Primary Key:** Department\_ID

**Attributes:**

- Department\_ID (Primary Key)
- Department\_Name (Unique)
- Location

## Entity 2: Employee

**Primary Key:** SSN

**Attributes:**

- SSN (Primary Key)
- Name
- Address
- Salary
- Sex
- Birth\_Date

## Entity 3: Project

**Primary Key:** Project\_Number

**Attributes:**

- Project\_Number (Primary Key)
- Project\_Name (Unique)
- Location

## Entity 4: Dependent (Weak Entity)

**Partial Key:** Dependent\_Name

**Attributes:**

- Dependent\_Name (Partial Key)
- Birth\_Date
- Relationship

**Dependent** is a **Weak Entity** because it cannot exist without an Employee.

# 3. Relationships

## 1. Manages

- One employee manages one department.
- Every department has one manager.
- Manager's **Start\_Date** is stored in this relationship.
- **Cardinality: 1 : 1**

**Relationship Attribute:**

- Start\_Date

**2. Works\_For**

- One department has many employees.
- Every employee belongs to only one department.
- **Cardinality: 1 : M**

**3. Controls**

- One department controls many projects.
- Every project belongs to one department.
- **Cardinality: 1 : M**

**4. Works\_On**

- One employee can work on many projects.
- One project can have many employees.
- **Cardinality: M : N**

**Relationship Attribute:**

- Hours\_Per\_Week

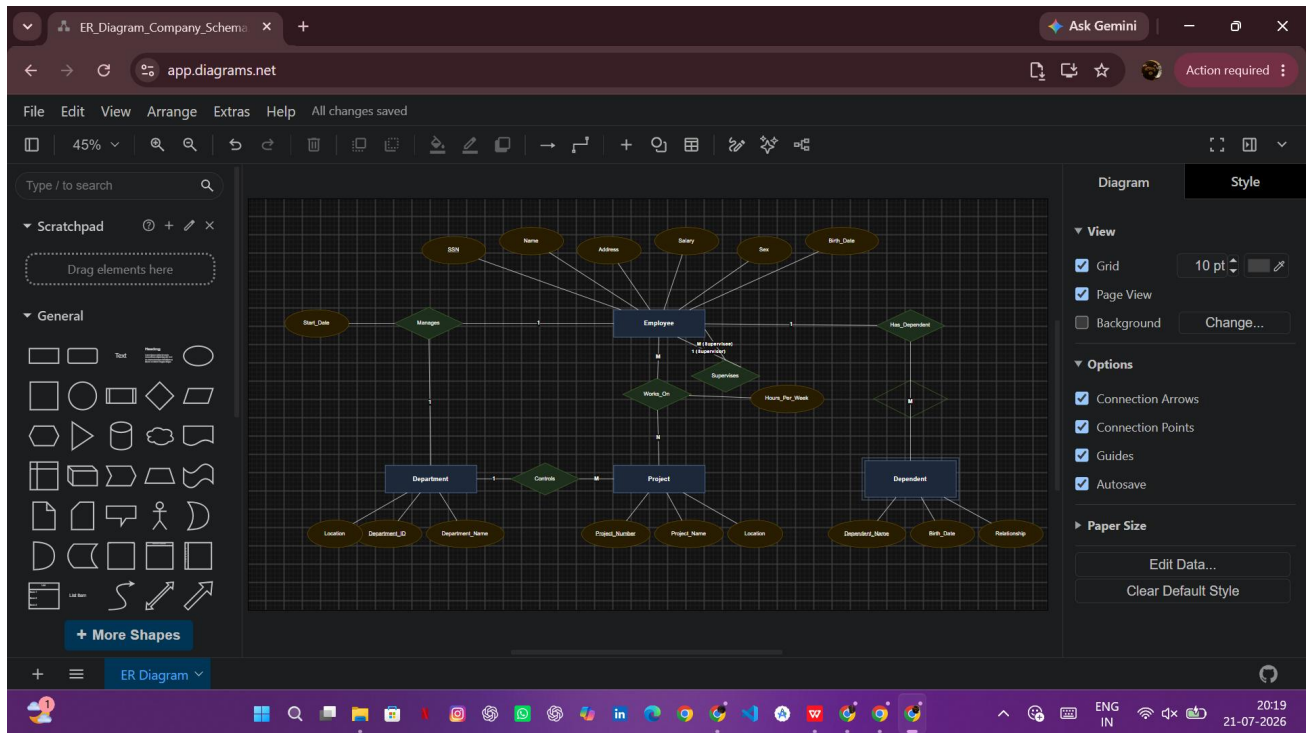
**5. Supervises**

- One employee supervises many employees.
- Each employee has only one immediate supervisor.
- **Cardinality: 1 : M** (Recursive Relationship)

**6. Has\_Dependent**

- One employee can have many dependents.
- Each dependent belongs to one employee.
- **Cardinality: 1 : M**

## 4. Complete ER Diagram



## 5. Cardinality Summary

| Relationship                     | Cardinality |
|----------------------------------|-------------|
| Department → Employee            | 1 : M       |
| Department → Project             | 1 : M       |
| Employee ↔ Project               | M : N       |
| Employee → Dependent             | 1 : M       |
| Employee → Employee (Supervisor) | 1 : M       |
| Employee ↔ Department (Manager)  | 1 : 1       |

## 6. Relational Schema

### Department

```
Department(  
    Department_ID (PK),  
    Department_Name,  
    Location  
)
```

### Employee

```
Employee(  
    SSN (PK),  
    Name,  
    Address,  
    Salary,  
    Sex,  
    Birth_Date,  
    Department_ID (FK),  
    Supervisor_SSN (FK)  
)
```

### Project

```
Project(  
    Project_Number (PK),  
    Project_Name,  
    Location,  
    Department_ID (FK)  
)
```

### Works\_On

```
Works_On(  
    SSN (FK),  
    Project_Number (FK),  
    Hours_Per_Week  
)
```

### Manages

```
Manages(  
    SSN (FK),  
    Department_ID (FK),  
    Start_Date  
)
```

### Dependent

```
Dependent(  

```

SSN (FK),  
Dependent\_Name,  
Birth\_Date,  
Relationship  
)

## 7. Conclusion

The Company Project Management System ER model consists of four main entities: **Department**, **Employee**, **Project**, and **Dependent**. It includes key relationships such as **Manages**, **Works\_For**, **Controls**, **Works\_On**, **Supervises**, and **Has\_Dependent**. The model accurately represents project assignments, department management, employee supervision, and dependent information while maintaining data integrity and minimizing redundancy. This design is suitable for efficient database implementation in a real-world company management system.



## Q6. Explain Schema and Instance with Examples. Distinguish between Logical Schema, Physical Schema, and View Schema.

### 1. Introduction

A **Database Management System (DBMS)** organizes data using different levels of schemas. A **schema** defines the structure of the database, while an **instance** represents the actual data stored in the database at a particular point in time.

### 2. Schema

#### Definition

A **schema** is the overall logical design or blueprint of a database. It specifies the database structure, including tables, attributes, relationships, constraints, and data types.

#### Characteristics

- Defines the database structure.
- Created during database design.
- Changes rarely.
- Also known as **database blueprint**.

#### Example

Consider a **Student** table:

```
Student(  
  Student_ID INT PRIMARY KEY,  
  Name VARCHAR(50),  
  Department VARCHAR(30),  
  CGPA DECIMAL(3,2)  
)
```

The above table definition represents the **schema** because it specifies the structure of the database.

### 3. Instance

## Definition

An **instance** is the actual collection of data stored in the database at a specific point in time.

## Characteristics

- Represents the current contents of the database.
- Changes frequently when records are inserted, updated, or deleted.
- Also known as the **database state**.

## Example

| Student_ID | Name  | Department | CGPA |
|------------|-------|------------|------|
| 101        | Rahul | CSE        | 8.75 |
| 102        | Priya | AI & ML    | 9.10 |
| 103        | Arjun | IT         | 8.45 |

The above records represent one **instance** of the database.

## 4. Difference Between Schema and Instance

| Schema                              | Instance                           |
|-------------------------------------|------------------------------------|
| Defines the database structure.     | Represents the actual data stored. |
| Changes rarely.                     | Changes frequently.                |
| Created during database design.     | Updated whenever data changes.     |
| Also called the database blueprint. | Also called the database state.    |

## 5. Types of Schema

DBMS follows the **Three-Schema Architecture**, which consists of:

1. Logical Schema

2. Physical Schema
3. View (External) Schema

## A. Logical Schema

### Definition

The **Logical Schema** describes the logical structure of the entire database, including tables, attributes, relationships, and constraints, without specifying how the data is physically stored.

### Characteristics

- Represents the conceptual design of the database.
- Independent of storage details.
- Used by database designers and developers.

### Example

```
Employee(  
  Emp_ID,  
  Name,  
  Salary,  
  Department  
)
```

## B. Physical Schema

### Definition

The **Physical Schema** describes how the data is actually stored in the storage devices, including file organization, indexing, and storage allocation.

### Characteristics

- Defines physical storage details.
- Includes indexes and storage methods.
- Managed by the DBMS.
- Used by database administrators (DBAs).

### Example

- Data stored in disk blocks.
- B+ Tree indexes.
- Hash files.
- File organization techniques.

## C. View Schema (External Schema)

### Definition

A **View Schema** defines how different users see the database. It displays only the required portion of the database and hides unnecessary or sensitive information.

### Characteristics

- Provides customized views for users.
- Enhances data security.
- Simplifies database access.
- Multiple view schemas can exist for one database.

### Example

#### Student Table

| Student_ID | Name  | Department | CGPA |
|------------|-------|------------|------|
| 101        | Rahul | CSE        | 8.75 |
| 102        | Priya | AI & ML    | 9.10 |

#### Faculty View

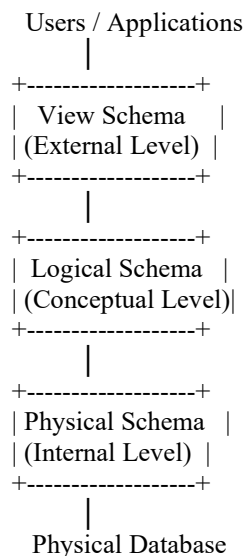
| Student_ID | Name  | CGPA |
|------------|-------|------|
| 101        | Rahul | 8.75 |
| 102        | Priya | 9.10 |

Only the required information is displayed.

## 6. Difference Between Logical, Physical, and View Schema

| Logical Schema                                   | Physical Schema                              | View Schema                            |
|--------------------------------------------------|----------------------------------------------|----------------------------------------|
| Describes the logical structure of the database. | Describes how data is physically stored.     | Describes how users view the database. |
| Independent of storage.                          | Depends on storage devices.                  | Independent of physical storage.       |
| Used by database designers.                      | Used by DBAs.                                | Used by end users and applications.    |
| Defines tables, attributes, and relationships.   | Defines files, indexes, and storage methods. | Displays only required data.           |
| One logical schema exists.                       | One physical schema exists.                  | Multiple view schemas can exist.       |

## 7. Three-Schema Architecture Diagram



## 8. Conclusion

A **schema** defines the structure of a database, whereas an **instance** represents the data stored at a particular moment. The **Logical Schema** describes the overall database structure, the **Physical Schema** explains how data is stored internally, and the **View Schema** provides customized views for different users. Together, these three schemas form the **Three-Schema Architecture**, ensuring data abstraction, security, and efficient database management.

